

6/6 Dynamic Programming →

- Algorithmic technique that solves complex problems by breaking them down into smaller sub-problems, storing their solutions for future use

eg

fib[0]	fib[1]	fib[2]	fib[3]	fib[4]	fib[5]
0	1	1	2	3	5

→ fib[5] is stored for future usage

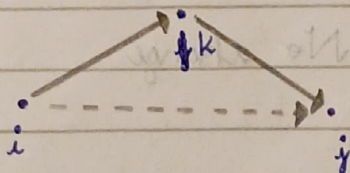
- Solve each subproblem only once
- Storing results in table
- Avoids redundant computations

Few Algorithms →

- Marshall's [transitive closure]
- Floyd's [shortest path between pair of nodes]
- Knapsack or 1/0 Knapsack
- Travelling salesman
- Bellman-Ford [shortest path from single source]

Marshall's Algorithm →

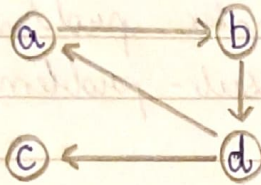
- Used to construct transitive closure for given directed graph



if (path[i][k] = 1 & path[k][j] = 1)
path[i][j] = 1

$$\therefore T(n) \in O(n^3)$$

eg



$$R(0) = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$R(a) = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} \rightarrow a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

Sol: $(d, a) = 1 \Rightarrow (d, b) = 1$
 Row: $(a, b) = 1$
 $(d, a) \&\& (a, b) = (d, b)$

$$R(b) = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} \rightarrow b \\ a \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

Sol: $(a, b) = 1, (d, b) = 1$
 Row: $(b, d) = 1$
 $\Rightarrow (a, d) = 1, (d, d) = 1$

$$R(c) = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} \rightarrow c \\ a \\ b \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

Sol: $(d, c) = 1$
 Row: -
 $\Rightarrow$ No change

	a	b	c	d		
$R(d) =$	a	0	1	0	1	Col: $(a,d)=1, (b,d)=1, (d,d)=1$
	b	0	0	0	1	Row: $(d,a)=1, (d,b)=1, (d,c)=1, (d,d)=1$
	c	0	0	0	0	
$\rightarrow d$		1	1	1	1	$\Rightarrow (a,a)=1, (a,b)=1, (a,c)=1$ $(b,a)=1, (b,b)=1, (b,c)=1$

Note (d,d) can be ignored

	a	b	c	d		
$R =$	a	1	1	1	1	[Transitive closure matrix]
	b	1	1	1	1	
	c	0	0	0	0	
	d	1	1	1	1	

ALGORITHM warshall($A[1 \dots n, 1 \dots n]$) $\rightarrow$

Purpose Computing transitive closure using Warshall's algo.

Input Adjacency matrix A of digraph & n nodes

Output Transitive closure of graph

Algo.

for $k = 0$ to $n-1$

for $i = 0$ to $n-1$

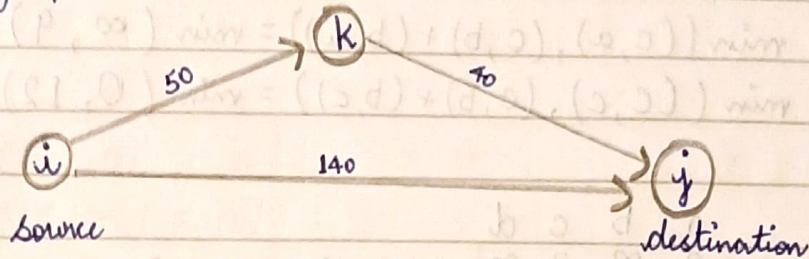
for $j = 0$ to $n-1$

if $(A[i][j] \parallel A[i][k] \&\& A[k][j])$

$A[i][j] = 1$

11/6 Floyd's Algorithm →

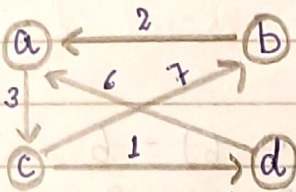
- Finds shortest path between pair of nodes for given weighted graph



$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

$$\therefore T(n) \in O(n^3)$$

eg



$$R(0) = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

Note: For $R(x)$, x should act as intermediate

- pick values other than 0 & ∞
- $(x, x) = 0$

$$R(a) = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & \infty & 3 & \infty \\ 2 & 0 & \infty & \infty \\ \infty & 7 & 0 & 1 \\ 6 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

Col: $(b, a) = 2, (d, a) = 6$
 Row: $(a, c) = 3$

$$(b, c) = \min((b, c), (b, a) + (a, c)) = \min(\infty, 2 + 3) = \min(\infty, 5) = 5$$

$$(d, c) = \min((d, c), (d, a) + (a, c)) = \min(\infty, 6 + 3) = \min(\infty, 9) = 9$$

$$R(b) = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 0 & \infty & 3 & \infty \\ b & 2 & 0 & 5 & \infty \\ c & \infty & 7 & 0 & 1 \\ d & 6 & \infty & 9 & 0 \end{array}$$

$$\text{Sol: } (c, b) = 7$$

$$\text{Row: } (b, a) = 2, (b, c) = 5$$

$$(c, a) = \min((c, a), (c, b) + (b, a)) = \min(\infty, 9) = 9$$

$$(c, c) = 0 \quad (c, c) = \min((c, c), (c, b) + (b, c)) = \min(0, 12) = 0$$

$$R(c) = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 0 & \infty & 3 & \infty \\ b & 2 & 0 & 5 & \infty \\ c & 9 & 7 & 0 & 1 \\ d & 6 & \infty & 9 & 0 \end{array}$$

$$\text{Sol: } (a, c) = 3, (b, c) = 5, (d, c) = 9$$

$$\text{Row: } (c, b) = 7, (c, d) = 1, (c, a) = 9$$

$$(a, b) = \min((a, b), (a, c) + (c, b)) = \min(\infty, 10) = 10$$

$$(a, d) = \min((a, d), (a, c) + (c, d)) = \min(\infty, 4) = 4$$

$$(a, a) = 0$$

$$(b, b) = 0$$

$$(b, d) = \min((b, d), (b, c) + (c, d)) = \min(\infty, 6) = 6$$

$$(b, a) = \min((b, a), (b, c) + (c, a)) = \min(2, 14) = 2$$

$$(d, b) = \min((d, b), (d, c) + (c, b)) = \min(\infty, 16) = 16$$

$$(d, d) = 0$$

$$(d, a) = \min((d, a), (d, c) + (c, a)) = \min(6, 18) = 6$$

$$R(d) = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 0 & 10 & 3 & 4 \\ b & 2 & 0 & 5 & 6 \\ c & 9 & 7 & 0 & 1 \\ d & 6 & 16 & 9 & 0 \end{array}$$

$$\text{Sol: } (a, d) = 4, (b, d) = 6, (c, d) = 1$$

$$\text{Row: } (d, a) = 6, (d, b) = 16, (d, c) = 9$$

$$(a, a) = 0$$

$$(a, b) = \min((a, b), (a, d) + (d, b)) = \min(10, 20) = 10$$

$$(a, c) = \min((a, c), (a, d) + (d, c)) = \min(3, 13) = 3$$

$$(b, a) = \min((b, a), (b, d) + (d, a)) = \min(2, 12) = 2$$

$$(b, b) = 0$$

$$(b,c) = \min((b,c), (b,d) + (d,c)) = \min(5, 15) = 5$$

$$(c,a) = \min((c,a), (c,d) + (d,a)) = \min(9, 7) = 7$$

$$(c,b) = \min((c,b), (c,d) + (d,b)) = \min(7, 17) = 7$$

$$(c,c) = 0$$

	a	b	c	d
R = a	0	10	3	4
b	2	0	5	6
c	7	7	0	1
d	6	16	9	0

12/6 ALGORITHM Floyd ($\text{dist}[\][\], n$) $\rightarrow$

Purpose Compute shortest path between all pairs for given weighted graph

Input $\text{dist}[\][\] \rightarrow$ weighted matrix of $n \times n$

Output $\text{dist}[\][\] \rightarrow$ Shortest path between all pairs

Algo.

for $k = 0$ to $n - 1$

for $i = 0$ to $n - 1$

for $j = 0$ to $n - 1$

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$$

Analysis $\rightarrow$

• Input size $\rightarrow$

• Basic operation $\rightarrow \text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$

$$\begin{aligned}
 T(n) &= \sum_{k=0}^{n-1} \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 = \sum_{k=0}^{n-1} \sum_{i=0}^{n-1} (n-1-0+1) = n \sum_{k=0}^{n-1} \sum_{i=0}^{n-1} 1 \\
 &= n \sum_{k=0}^{n-1} (n-1-0+1) = n^2 \sum_{k=0}^{n-1} 1 = n^2 (n-1-0+1) \\
 &= n^3
 \end{aligned}$$

$$\therefore T(n) \in O(n^3)$$

1/0 Knapsack Problem →

- Main object is to place item into knapsack (bag) so that maximum profit is obtained
- Weight of chosen item should not exceed bag capacity

kg	Max. capacity of bag $m = 5$ kg	
Item no.	Weight	Profit
1	2kg (onion)	\$12
2	1kg (tomato)	\$10
3	3kg (carrrot)	\$20
4	2kg (potato)	\$15

Here, $m = 5$, $n = 4$

			0	1	2	3	4	5 (m)
Weight []	Profit []	0	0	0	0	0	0	0
2kg	\$12	1	0	0	12	12	12	12
1kg	\$10	2	0	10	12	22	22	22
3kg	\$20	3	0	10	12	22	30	32
2kg	\$15	4	0	10	15	25	30	37
		(n)						

Optimal solution : Max profit = \$37

[2kg onion + 1kg tomato + 2kg potato = 5kg]

ALGORITHM $\text{knapsack}(\text{weight}[], \text{profit}[], n, m) \rightarrow$

Purpose Optimal solution to given problem
Input $\text{weight}[] \rightarrow$ weight of each item (n items)
 $\text{profit}[] \rightarrow$ profit of each item (n items)
 $n \rightarrow$ no. of items
 $m \rightarrow$ max. capacity of bag

Output Returns optimal solution $V[n][m]$

Algo.

```

for i = 0 to n
  for j = 0 to m
    if (i == 0 || j == 0)
      V[i][j] = 0
    else if (weight[i] > j)
      V[i][j] = V[i-1][j]
    else
      V[i][j] = max(profit[i] + V[i-1][j - weight[i]], V[i-1][j])
  return V[n][m]

```

Analysis $\rightarrow$

- Input size $\rightarrow$
- Basic operation $\rightarrow$

$$\begin{aligned}
 T(n) &= \sum_{i=0}^n \sum_{j=0}^m 1 = \sum_{i=0}^n (m - 0 + 1) = \sum_{i=0}^n (m + 1) = (m + 1) \sum_{i=0}^n 1 \\
 &= (m + 1)(n - 0 + 1) = (m + 1)(n + 1) = mn + 1 + m + n \\
 &\approx mn
 \end{aligned}$$

$$\therefore T(n) \in O(mn)$$

eg Weight (2kg, 4kg, 2kg, 1kg, 3kg)

profit (\$30, \$25, \$40, \$15, \$30)

Max. capacity of container = 6kg

			0	1	2	3	4	5	6
weight []	profit []		0	0	0	0	0	0	0
2kg	\$30	1	0	0	30	30	30	30	30
4kg	\$25	2	0	0	30	30	30	30	55
2kg	\$40	3	0	0	40	40	70	70	70
1kg	\$15	4	0	15	40	55	70	85	85
3kg	\$30	5	0	15	40	55	70	85	85

(n)

Max profit = \$85 [2kg (\$30) + 2kg (\$40) + 1kg (\$15)]

13/6 Greedy Technique →

- Simple & straightforward way to solve optimization problems
- Algo. that finds solution to problem in shortest time possible

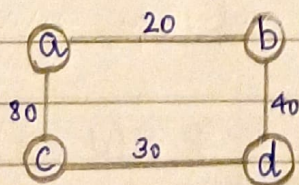
Few Algorithms →

- Kruskal's algo.
- Prim's algo.
- Dijkstra's algo.
- Huffman Trees
- Knapsack problem
- Travelling Salesman problem

Spanning Tree →

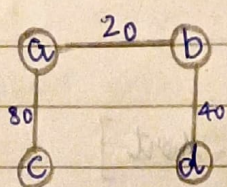
- Tree in which all nodes are connected without forming cycles
- Graph $G = (V, E)$ should be connected & acyclic

eg Find all possible spanning trees for:

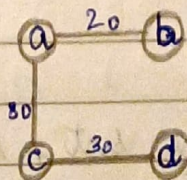


$$|E| = |V| - 1$$

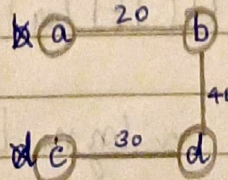
$$\text{Here, } 3 = 4 - 1$$



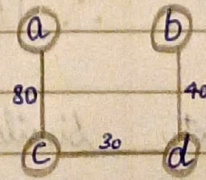
Spanning Tree
I



Spanning Tree
II



Spanning Tree
III



Spanning Tree
IV

MST Minimum spanning tree $\rightarrow$

- Spanning tree (connected & acyclic), but with minimum cost

eg In previous example, minimum spanning tree is spanning tree III

$$ST I = 80 + 20 + 40 = 140$$

$$ST II = 80 + 20 + 30 = 130$$

$$ST III = 20 + 30 + 40 = 90 \quad (\text{minimum})$$

$$ST IV = 80 + 30 + 40 = 150$$

Applications of MST $\rightarrow$

- Computer networking
- AI network
- Telephone network
- TV cable

To obtain MST :

- Kruskal's
- Prim's

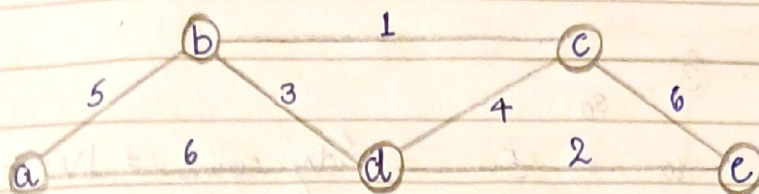
Kruskal's Algorithm $\rightarrow$

- Used to find MST
- Based on greedy technique

$$\therefore T(n) \in O(|E| \log |E|)$$

Note Similar to $n \log n$ [merge sort, quick sort]

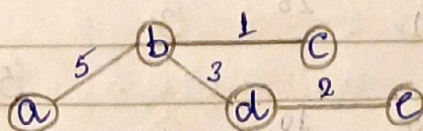
eg



(b,c) (d,e) (b,d) (c,d) (b,a) (a,d) (c,e)
 1 2 3 4 5 6 6

	Spanning Tree Construction	Count edge
Create forest		$ne = 0$
(b,c) = 1		$ne = 1$
(d,e) = 2		$ne = 2$
(d,b) = 3		$ne = 3$
(c,d) = 4		$ne = 3$
(a,b) = 5		$ne = 4$

MST →



$$\text{Cost} = 1 + 2 + 3 + 5 = 11$$

19/6
eg

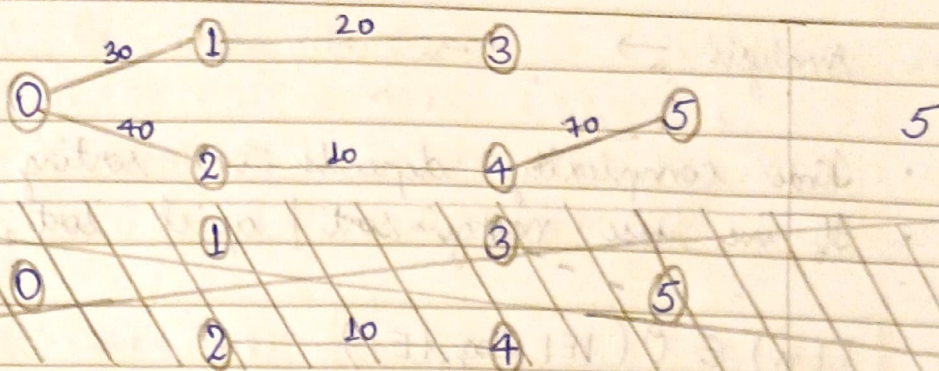
$$\text{Edge count} = |V| - 1$$

$$5 = 6 - 1$$

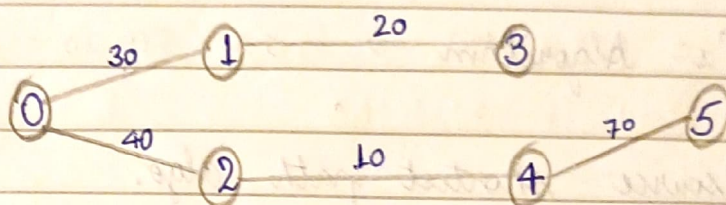
(2,4)	(1,3)	(0,1)	(0,2)	(1,2)	(3,4)	(4,5)	(3,5)
10	20	30	40	50	60	70	80

Minimum Edge	Constructing MST		Edge count
Create forest	0	1 3 4 5	0
(2,4) = 10	0	1 3 4 5 2 — 10 — 4	1
(1,3) = 20	0	1 3 4 5 2 — 10 — 4 1 — 20 — 3	2
(0,1) = 30	0	1 3 4 5 2 — 10 — 4 1 — 20 — 3 0 — 30 — 1	3
(0,2) = 40	0	1 3 4 5 2 — 10 — 4 1 — 20 — 3 0 — 30 — 1 0 — 40 — 2	4
(1,2) = 50	0	1 3 4 5 2 — 10 — 4 1 — 20 — 3 0 — 30 — 1 0 — 40 — 2 1 — 50 — 2	4 (rejected)
(3,4) = 60	0	1 3 4 5 2 — 10 — 4 1 — 20 — 3 0 — 30 — 1 0 — 40 — 2 1 — 50 — 2 3 — 60 — 4	4 (rejected)

$$(4,5) = 70$$



MST $\rightarrow$



$$\text{Cost} = 10 + 20 + 30 + 40 + 70 = 170$$

ALGORITHM $\text{kruskal}(G) \rightarrow$

Purpose Constructing MST using Kruskal's algo.

Input $G = (V, E) \rightarrow$ Weighted Connected graph

Output Return MST E_{MST}

Algo. $E_{\text{MST}} = \emptyset$ // Initialize set of MST edges to null set

$E_{\text{Count}} = 0$ // Edge count initialized to zero

Sort all edges in ascending order for given weighted graph

while ($E_{\text{Count}} < |V| - 1$)

 Select minimum edge E_{uv} from sorted list

 if ($E_{\text{MST}} \cup E_{uv}$ is acyclic)

$E_{\text{MST}} = E_{\text{MST}} \cup E_{uv}$

$E_{\text{Count}} = E_{\text{Count}} + 1$

// Connected edge (u,v)

endwhile

return E_{MST}

Analysis →

- Time complexity depends on sorting all edges
- If we use merge sort / quick sort,

$$T(n) \in O(|E| \log_2 |E|)$$

20/6 Dijkstra's Algorithm →

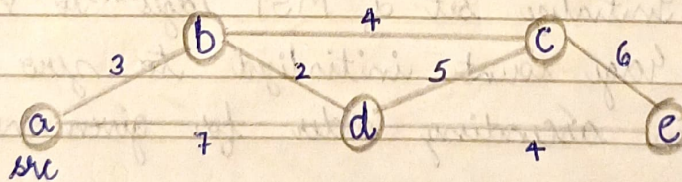
- Single source shortest path algo.
- Based on greedy technique

$$\therefore T(n) \in O(|E| \log_2 |V|)$$

To obtain single source shortest path:

- Dijkstra (greedy) (works only on +ve edges)
- Bellman - Ford (dynamic) (works on +ve & -ve edges)

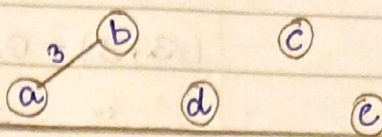
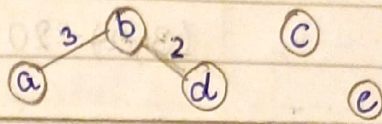
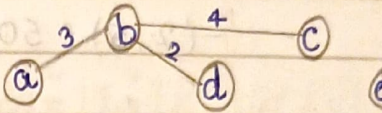
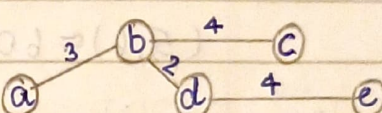
eg Shortest distance from given source a,



Initial →

	a	b	c	d	e
visited []	0	0	0	0	0
a	b	c	d	e	
(a,a)	visited []	1	0	0	0

Note Stop, when all are 1s

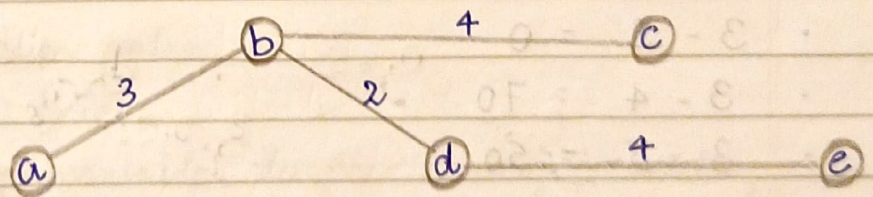
Tree Vertices	Remaining Vertices	Shortest path
$(a, a) = 0$	$(a, b) = 0 + 3 = 3$; $(a, c) = 0 + \infty = \infty$ $(a, d) = 0 + 7 = 7$; $(a, e) = 0 + \infty = \infty$	
$(a, b) = 3$	$(b, c) = 3 + 4 = 7$; $(b, d) = 3 + 2 = 5$ $(a, e) = \infty$	
$(b, d) = 5$	$(b, c) = 7$; $(d, e) = 5 + 4 = 9$	
$(b, c) = 7$	$(d, e) = 9$	

All vertices are visited $\rightarrow$

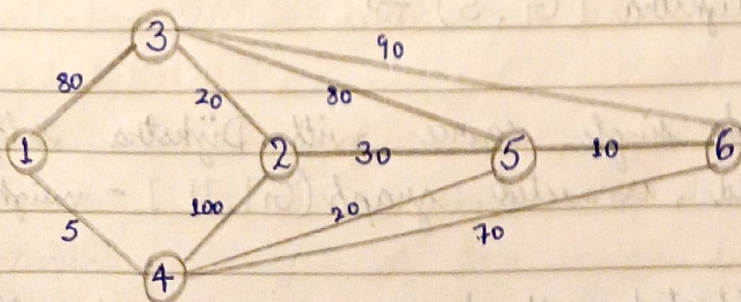
	a	b	c	d	e
visited []	1	1	1	1	1

Single source shortest path $\rightarrow$

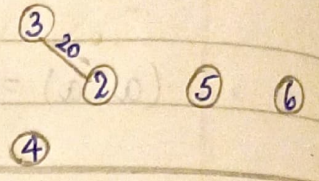
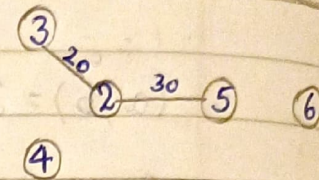
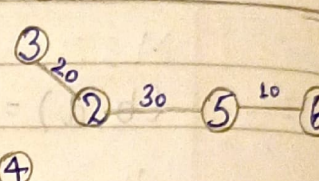
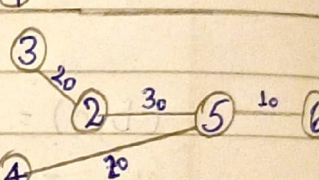
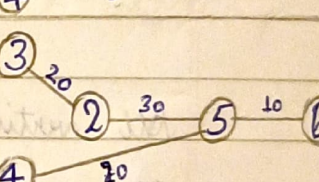
- $a - a = 0$
- $a - b = 3$
- $a - c = 7$
- $a - d = 5$
- $a - e = 9$



eg

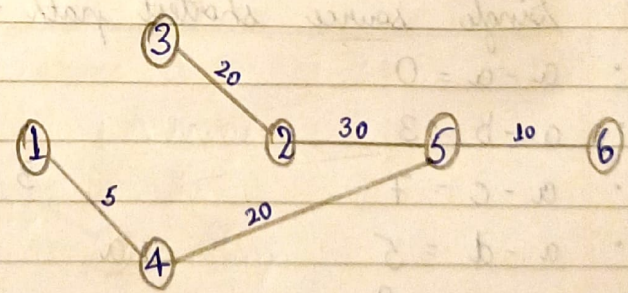


Source $\rightarrow 3$

Free Vertices	Remaining Vertices	Shortest path
$(3,1) = 0$ $(3,3) = 0$	$(3,1) = 0 + 80 = 80$; $(3,2) = 0 + 20 = 20$ $(3,4) = 0 + \infty = \infty$; $(3,5) = 0 + 80 = 80$ $(3,6) = 0 + 90 = 90$	
$(3,2) = 20$	$(3,1) = 80$; $(2,4) = 20 + 100 = 120$ $(2,5) = 20 + 30 = 50$; $(3,6) = 90$	
$(2,5) = 50$	$(3,1) = 80$; $(5,4) = 50 + 20 = 70$ $(5,6) = 50 + 10 = 60$	
$(5,6) = 60$	$(3,1) = 80$; $(5,4) = 70$	
$(5,4) = 70$	$(4,1) = 70 + 5 = 75$	

Single source shortest path $\rightarrow$

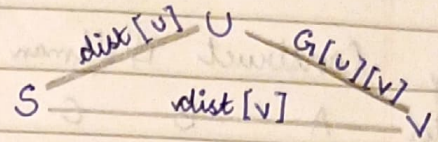
- $3 - 1 = 75$
- $3 - 2 = 20$
- $3 - 3 = 0$
- $3 - 4 = 70$
- $3 - 5 = 50$
- $3 - 6 = 60$



21/6 ALGORITHM by dijkstra (G, S) $\rightarrow$

Purpose Find shortest path of single source with Dijkstra algo.
 Input $G = (V, E) \rightarrow$ Weighted, connected graph ($G[i][j]$ - weighted matrix)
 $S \rightarrow$ Source vertex
 Output Return $dist[] \rightarrow$ Shortest path from source node to all other nodes

Algo. for $i = 0$ to $|V| - 1$
 $\text{dist}[i] = \infty$ // Initialize source to all nodes with ∞
 $\text{visited}[i] = 0$ // Initialize all nodes as unvisited
 $\text{dist}[S] = 0$
 $\text{visited}[S] = 1$
 for $\text{ECount} = 0$ to $|V| - 1$
 $u = \min(\text{dist})$
 $\text{visited}[u] = 1$
 for $v = 0$ to $|V| - 1$
 if ($\text{visited}[v] == 0$ & $\text{dist}[u] + G[u][v] < \text{dist}[v]$)
 $\text{dist}[v] = \text{dist}[u] + G[u][v]$
 return dist



Analysis $\rightarrow$

$$\therefore T(n) \in \Theta(|E| \log_2 |V|)$$

Huffman Trees & Code $\rightarrow$

- Lossless data compression algo.
- Based on greedy technique
- Best possible way of variable length binary encoding for given characters & frequencies

- Step I. Initialize n one-node trees & label them with characters of alphabet
- Record frequency of each character in its tree's root to indicate tree's weight
 - Generally, weight of tree will be equal to sum of frequencies in tree's leaves

- Step II: Repeat following operation until single tree is obtained
- Find two trees with smallest weight (ties can be broken arbitrarily, but ~~not~~ when required)
 - Make them left & right subtree

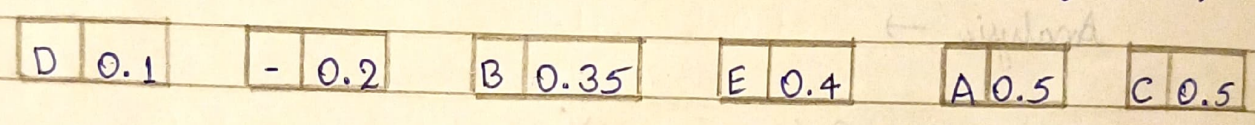
Eg Construct Huffman tree & obtain its Huffman code:

Character	A	B	C	D	E	-
Probability	0.5	0.35	0.5	0.1	0.4	0.2

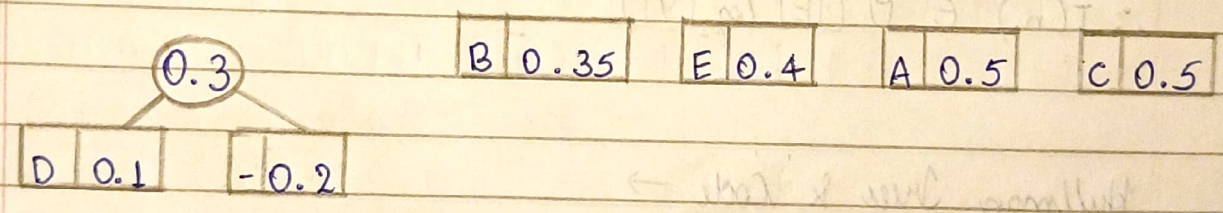
i) Encode text DAD = BE

ii) Decode text 1100110110

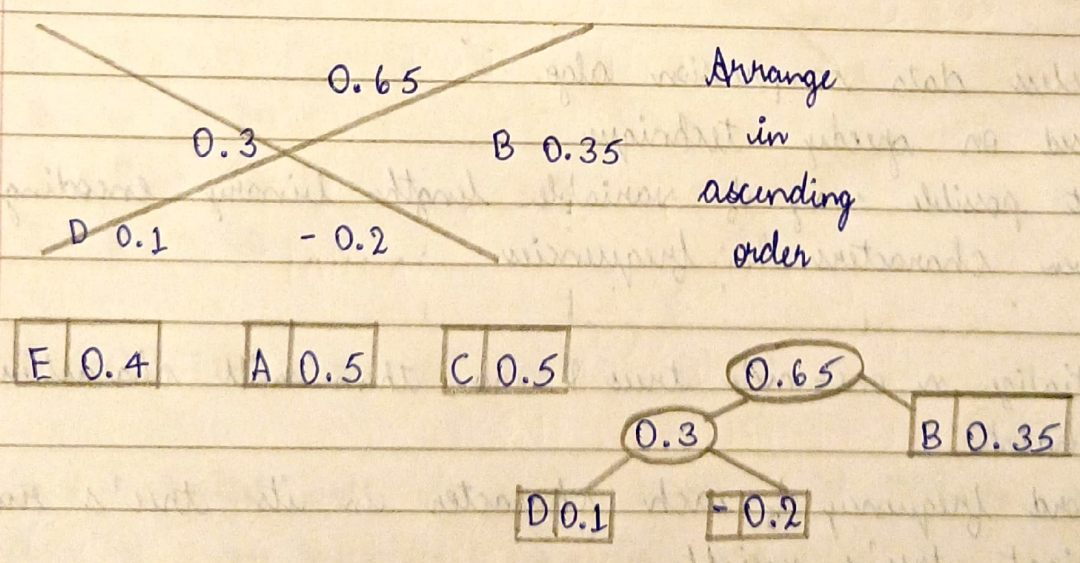
Step I Create node for each character attached with frequency



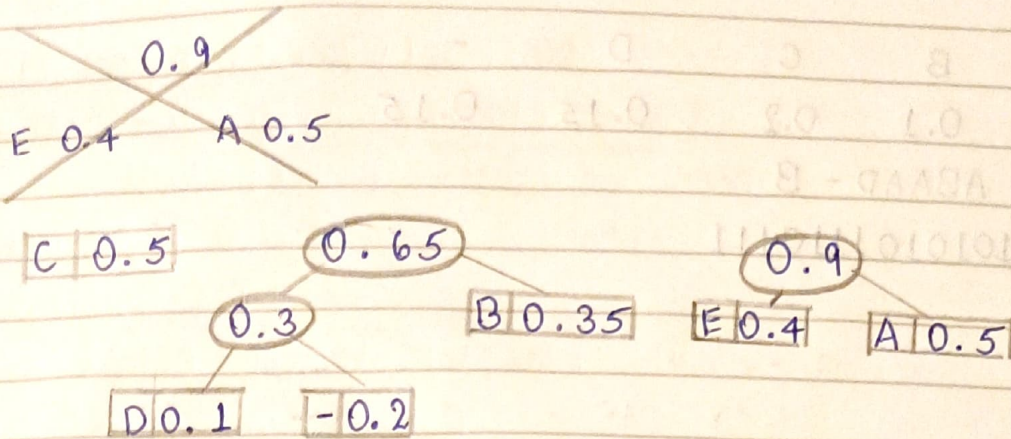
Step II



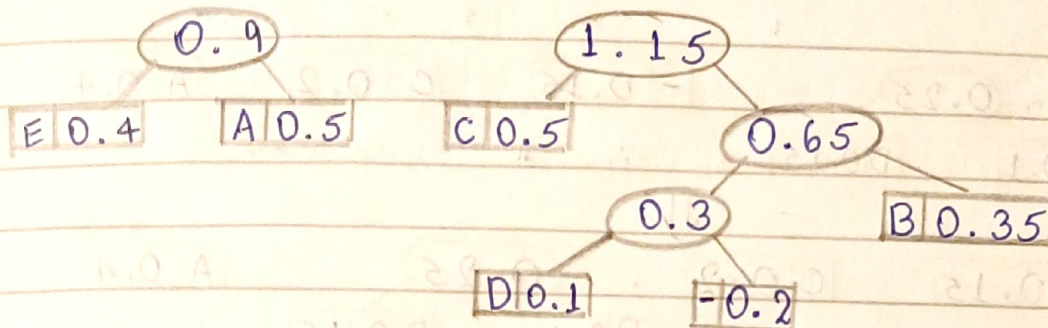
Step III



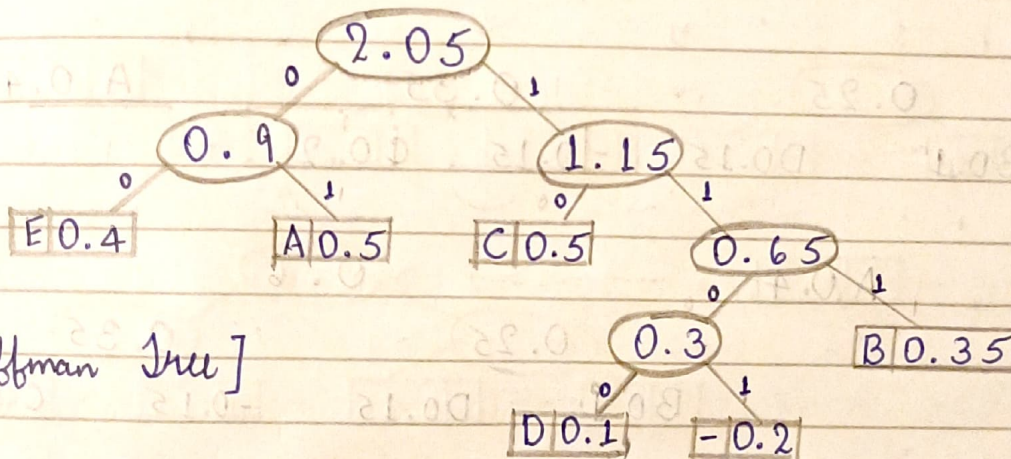
Step IV



Step V



Step VI



[Huffman Tree]

Character	A	B	C	D	E	-
Huffman Code	01	111	10	1100	00	1101

Solutions :

i) 1100 01 1100 1101 111 EE

ii) D - C [1100 110110]

eg A B C D -
 0.4 0.1 0.2 0.15 0.15

i) Encode ABAAD - B

ii) Decode 1010101110111

Tree →

Step I B 0.1 D 0.15 - 0.15 C 0.2 A 0.4

Step II 0.25 - 0.15 C 0.2 A 0.4
 B 0.1 D 0.15

↓
- 0.15 C 0.2 0.25 A 0.4
 B 0.1 D 0.15

Step III 0.25 0.35 A 0.4
 B 0.1 D 0.15 - 0.15 C 0.2

Step IV A 0.4 0.6 0.25 0.35
 B 0.1 D 0.15 - 0.15 C 0.2

Step V

```

graph TD
    Root((1)) -- 0 --> A[A 0.4]
    Root -- 1 --> Node0.6((0.6))
    Node0.6 -- 0 --> Node0.25((0.25))
    Node0.6 -- 1 --> Node0.35((0.35))
    Node0.25 -- 0 --> B[B 0.1]
    Node0.25 -- 1 --> D[D 0.15]
    Node0.35 -- 0 --> Minus[- 0.15]
    Node0.35 -- 1 --> C[C 0.2]
  
```

Code →

A	B	C	D	-
0	100	111	101	110

- i) 0 100 0 0 101 110 100
- ii) DAD - C [10 10 10 111 0111]